JavaScript Variable Scope — Complete Teaching Reference

1. What is Scope?

Scope defines where a variable can be accessed in a JavaScript program.

In simple words:

> Scope controls the visibility and accessibility of variables.

```
let name = "Udit";

function greet() {
  console.log(name);
}

greet();
```

Output:

```
Udit
```

The function can access `name` because `name` exists in an outer scope.

---

2. Main Types of Scope in JavaScript

JavaScript mainly has these scopes:

1. Global Scope
2. Function Scope
3. Block Scope
4. Module Scope

To understand how they work properly, also understand:

- Lexical Scope
- Scope Chain
- Variable Shadowing

- Temporal Dead Zone
- Closures

---

3. Global Scope

A variable declared outside all functions and blocks belongs to the **global scope**.

```
let course = "JavaScript";

function showCourse() {
  console.log(course);
}

showCourse();
console.log(course);
```

Output:

```
JavaScript
JavaScript
```

A global variable can usually be accessed from inner scopes.

```
let username = "Udit";

function first() {
  console.log(username);
}

function second() {
  console.log(username);
}

first();
second();
```

Output:

```
Udit
Udit
```

Important Point

Global variables should not be created unnecessarily because many parts of the program can access or modify them.

---

4. Function Scope

Variables declared inside a function are local to that function.

```
function test() {
  let message = "Hello";
  console.log(message);
}

test();
```

Output:

```
Hello
```

Trying to access the variable outside:

```
function test() {
  let message = "Hello";
}

console.log(message);
```

Output:

```
ReferenceError
```

Function Parameters Are Also Local

```
function greet(name) {
  console.log(name);
}

greet("Udit");

console.log(name);
```

Output:

```
Udit
ReferenceError
```

`name` belongs to the function's local scope.

---

5. var and Function Scope

`var` is **function scoped**.

```
function test() {
  var a = 10;

  console.log(a);
}

test();
```

Output:

```
10
```

Outside the function:

```
function test() {
  var a = 10;
}

console.log(a);
```

Output:

```
ReferenceError
```

---

6. Block Scope

A block is code written inside curly braces:

```
{
}
```

Examples of blocks:

```
if (true) {
}

for (;;) {
}

while (true) {
}
```

`let` and `const` are **block scoped**.

```
{
  let a = 10;
  const b = 20;

  console.log(a);
  console.log(b);
}

console.log(a);
```

Output:

```
10
20
ReferenceError
```

---

7. var is NOT Block Scoped

This is one of the most important scope differences.

```
{
  var a = 10;
}

console.log(a);
```

Output:

```
10
```

But:

```
{
  let b = 20;
}

console.log(b);
```

Output:

```
ReferenceError
```

And:

```
{
  const c = 30;
}

console.log(c);
```

Output:

```
ReferenceError
```

Core Rule

```
var   → Function Scoped
let   → Block Scoped
const → Block Scoped
```

---

8. Scope Inside `if`

Using `var`

```
if (true) {
  var message = "Hello";
}

console.log(message);
```

Output:

```
Hello
```

Using `let`

```
if (true) {
  let message = "Hello";
}

console.log(message);
```

Output:

```
ReferenceError
```

Using `const`

```
if (true) {
  const message = "Hello";
}

console.log(message);
```

Output:

```
ReferenceError
```

---

9. Scope Inside Loops

`let`

```
for (let i = 0; i < 3; i++) {
  console.log(i);
}

console.log(i);
```

Output:

```
0
1
2
ReferenceError
```

`var`

```
for (var i = 0; i < 3; i++) {
  console.log(i);
}

console.log(i);
```

Output:

```
0
1
2
3
```

Because `var` does not create block scope for the loop.

---

10. Nested Scope

A scope can exist inside another scope.

```
let a = 10;

function outer() {
  let b = 20;

  function inner() {
    let c = 30;

    console.log(a);
    console.log(b);
    console.log(c);
  }

  inner();
}

outer();
```

Output:

```
10
20
30
```

The inner function can access:

- its own variables
- variables from the outer function
- global variables

---

11. Inner Scope vs Outer Scope

Important rule:

```
Inner Scope → Can access Outer Scope

Outer Scope → Cannot access Inner Scope
```

Example:

```
function outer() {
  let a = 10;

  function inner() {
    let b = 20;

    console.log(a);
  }

  inner();

  console.log(b);
}

outer();
```

Output:

```
10
ReferenceError
```

---

12. Lexical Scope

JavaScript uses **lexical scoping**.

It means:

> A function's accessible variables are determined by where that function is written in the source code.

```
let name = "Global";

function outer() {
  let name = "Outer";

  function inner() {
    console.log(name);
  }

  inner();
}

outer();
```

Output:

```
Outer
```

`inner()` first looks for `name` inside its own scope.

It does not find it there, so it looks in its outer lexical scope and finds:

```
let name = "Outer";
```

---

13. Scope Chain

When JavaScript tries to find a variable, it searches through scopes from inside to outside.

```
Current Scope
     ↓
Parent Scope
     ↓
Parent's Parent Scope
     ↓
Global Scope
```

Example:

```
let a = 10;

function first() {
  let b = 20;

  function second() {
    let c = 30;

    console.log(a + b + c);
  }

  second();
}

first();
```

Output:

```
60
```

JavaScript finds:

```
c → Current Scope
b → Parent Scope
a → Global Scope
```

This lookup process is called the **scope chain**.

---

14. Variable Shadowing

A variable declared inside an inner scope can have the same name as a variable in an outer scope.

```
let name = "Global";

function test() {
  let name = "Local";

  console.log(name);
}

test();
console.log(name);
```

Output:

```
Local
Global
```

The local variable temporarily hides the outer variable.

This is called **variable shadowing**.

---

15. Block Shadowing

```
let age = 25;

{
  let age = 30;

  console.log(age);
}

console.log(age);
```

Output:

```
30
25
```

Both variables are different because they belong to different scopes.

---

16. Illegal Shadowing

Some shadowing combinations are not allowed.

```
let a = 10;

{
  var a = 20;
}
```

Output:

```
SyntaxError
```

Why?

`var` is not block scoped, so it conflicts with the existing `let`.

But this is allowed:

```
var a = 10;

{
  let a = 20;

  console.log(a);
}

console.log(a);
```

Output:

```
20
10
```

---

17. Same Variable Name in Different Functions

Different functions have separate local scopes.

```
function first() {
  let message = "First";
  console.log(message);
}

function second() {
  let message = "Second";
  console.log(message);
}

first();
second();
```

Output:

```
First
Second
```

No conflict occurs.

---

18. Scope and Hoisting with `var`

`var` declarations are hoisted inside their scope.

```
function test() {
  console.log(a);

  var a = 10;
}

test();
```

Output:

```
undefined
```

Conceptually JavaScript behaves approximately like:

```
function test() {
  var a;

  console.log(a);

  a = 10;
}
```

The declaration is available, but the assignment happens later.

---

19. Temporal Dead Zone with `let`

```
console.log(a);

let a = 10;
```

Output:

```
ReferenceError
```

The variable belongs to the scope, but it cannot be accessed before its declaration is executed.

The time between entering the scope and reaching the declaration is called the:

Temporal Dead Zone (TDZ)

---

20. Temporal Dead Zone with `const`

```
console.log(a);

const a = 10;
```

Output:

```
ReferenceError
```

`const` also has a Temporal Dead Zone.

---

21. Important TDZ Edge Case

```
let a = 10;

{
  console.log(a);

  let a = 20;
}
```

A beginner may expect:

```
10
```

Actual result:

```
ReferenceError
```

Why?

The block already has its own local `a` because of:

```
let a = 20;
```

That local variable shadows the outer `a`.

Before the declaration is reached, the local `a` is in the Temporal Dead Zone.

JavaScript does not fall back to the outer variable.

---

22. Function Scope and Block Scope Together

```
function test() {
  var a = 10;

  if (true) {
    let b = 20;
    const c = 30;

    console.log(a);
    console.log(b);
    console.log(c);
  }

  console.log(a);
  console.log(b);
}

test();
```

Output:

```
10
20
30
10
ReferenceError
```

Explanation:

```
a → Function Scoped
b → Block Scoped
c → Block Scoped
```

---

23. var Inside a Block Inside a Function

```
function test() {
  if (true) {
    var a = 10;
  }

  console.log(a);
}

test();
```

Output:

```
10
```

The `if` block does not create a new scope for `var`.

The variable belongs to the whole function.

---

24. `let` Inside a Block Inside a Function

```
function test() {
  if (true) {
    let a = 10;
  }

  console.log(a);
}

test();
```

Output:

```
ReferenceError
```

The variable exists only inside the `if` block.

---

25. Important Loop Edge Case with `var`

```
for (var i = 1; i <= 3; i++) {
  setTimeout(function () {
    console.log(i);
  }, 1000);
}
```

Output after the loop finishes:

```
4
4
4
```

Why?

All callbacks use the same `i`.

After the loop:

```
i = 4
```

So all callbacks print `4`.

---

26. Same Loop with `let`

```
for (let i = 1; i <= 3; i++) {
  setTimeout(function () {
    console.log(i);
  }, 1000);
}
```

Output:

```
1
2
3
```

For a `for` loop using `let`, JavaScript creates a separate binding for each iteration.

Conceptually:

```
Iteration 1 → i = 1
Iteration 2 → i = 2
Iteration 3 → i = 3
```

This is an important practical use of block scope.

---

27. Closures and Scope

A closure happens when a function remembers variables from the scope where it was created.

```
function outer() {
  let count = 0;

  function inner() {
    count++;
    console.log(count);
  }

  return inner;
}

const counter = outer();

counter();
counter();
counter();
```

Output:

```
1
2
3
```

Even after `outer()` has completed, the returned `inner()` function still has access to:

```
count
```

This is possible because of **closures and lexical scope**.

---

28. Module Scope

When JavaScript files are used as ES Modules, top-level variables stay inside that module unless exported.

`user.js`

```
const username = "Udit";

export function showUser() {
  console.log(username);
}
```

app.js

```
import { showUser } from "./user.js";

showUser();

console.log(username);
```

Output:

```
Udit
ReferenceError
```

`username` belongs to the module scope of `user.js`.

It is not automatically available inside `app.js`.

---

29. Browser Global Scope Edge Case

In a classic browser script:

```
var a = 10;
let b = 20;
const c = 30;

console.log(window.a);
console.log(window.b);
console.log(window.c);
```

Typically:

```
10
undefined
undefined
```

Top-level `var` declarations in a classic browser script can become properties of `window`.

Top-level `let` and `const` do not.

This behavior is different when JavaScript is running as an ES Module.

---

30. `catch` Block Scope

The variable created in a `catch` clause belongs to the `catch` block.

```
try {
  throw new Error("Something went wrong");
} catch (error) {
  console.log(error.message);
}

console.log(error);
```

Output:

```
Something went wrong
ReferenceError
```

`error` cannot be accessed outside the `catch` block.

---

31. Combined Practical Example

```
let username = "Guest";

function login() {
  let username = "Udit";

  if (true) {
    let message = "Login Successful";

    console.log(username);
    console.log(message);
  }

  console.log(username);
}

login();

console.log(username);
```

Output:

```
Udit
Login Successful
Udit
Guest
```

This single example demonstrates:

- Global Scope
- Function Scope
- Block Scope
- Variable Shadowing

---

32. Scope Summary

Declaration	Function Scoped	Block Scoped
`var`	Yes	No
`let`	Yes	Yes
`const`	Yes	Yes

Remember

```
var
↓
Function Scoped


let / const
↓
Block Scoped
```

---

33. Scope Lookup Rule

JavaScript searches for variables from the nearest scope outward.

```
Current Scope
     ↓
Outer Scope
     ↓
Outer Scope
     ↓
Global Scope
```

If JavaScript cannot find the variable anywhere in the accessible scope chain:

```
ReferenceError
```

---

34. Final Concepts to Remember

Scope

Determines where a variable is accessible.

Global Scope

Variables declared outside functions and blocks.

Function Scope

Variables local to a function.

Block Scope

Variables declared using `let` and `const` inside `{}`.

Lexical Scope

Variable access depends on where functions are written.

Scope Chain

JavaScript searches from the current scope toward outer scopes.

Shadowing

An inner variable can hide an outer variable with the same name.

Temporal Dead Zone

`let` and `const` cannot be accessed before their declarations execute.

Closure

A function can remember variables from the lexical scope where it was created.

Module Scope

Top-level variables inside ES Modules stay inside the module unless explicitly exported.

---

35. Final Mental Model

Whenever JavaScript encounters a variable:

```
console.log(value);
```

Think:

```
1. Is `value` available in the current scope?
              ↓ No

2. Is it available in the outer scope?
              ↓ No

3. Continue moving outward.
              ↓

4. Reach Global Scope.
              ↓

5. If still not found:
   ReferenceError
```

That is the foundation of JavaScript variable scope.